

LINUX

Training Documentation

An overview of the origin, architecture, and fundamentals of the Linux operating system, along with its core commands and concepts.

Table of Contents

1. Introduction to Linux
2. Origin and History of Linux
3. Linux Architecture
4. Features of Linux
5. Linux Distributions
6. The Linux File System
7. Basic Linux Commands
8. File and Directory Permissions
9. Users and Groups
10. Process Management
11. Package Management
12. Redirection, Pipes, and I/O
13. Shell Scripting Basics
14. Summary

1. Introduction to Linux

1.1 What is Linux?

- Linux is a free, open-source operating system kernel that forms the core of many complete operating systems, commonly called Linux distributions.
- It manages a computer's hardware resources — CPU, memory, storage, and devices — and allows software to run on top of it.
- It is widely used across servers, desktops, embedded systems, mobile devices (Android), and supercomputers.

1.2 Why Learn Linux?

- Powers the majority of the world's servers, cloud infrastructure, and websites.
- Open-source and free to use, study, modify, and distribute.
- Highly stable, secure, and customizable compared to many other operating systems.
- Essential skill for developers, system administrators, and DevOps engineers.

1.3 Where Is Linux Used?

Environment	Example Use Case
Servers	Web servers, database servers, and cloud infrastructure (most of the internet runs on Linux)
Desktops	Everyday computing via distributions like Ubuntu, Fedora, and Linux Mint
Mobile Devices	Android, the world's most-used mobile OS, is built on the Linux kernel
Embedded Systems	Routers, smart TVs, IoT devices, and automotive systems
Cloud & Containers	Almost all public cloud instances and container platforms like Docker run on Linux
Supercomputers	Virtually all of the world's top 500 supercomputers run Linux

2. Origin and History of Linux

2.1 The Unix Background

- In the late 1960s and 1970s, Unix was developed at Bell Labs by Ken Thompson and Dennis Ritchie.

- Unix introduced key ideas still used today — a hierarchical file system, multitasking, and multi-user support.
- Unix was proprietary, which encouraged the development of free alternatives.

2.2 The GNU Project

- In 1983, Richard Stallman started the GNU Project to build a completely free Unix-like operating system.
- The GNU Project produced many essential tools — compilers, editors, and utilities — but by the early 1990s still lacked a working kernel.

2.3 The Birth of Linux

- In 1991, Linus Torvalds, a Finnish computer science student, began developing a free kernel as a personal project.
- He announced it on the comp.os.minix newsgroup, describing it as "just a hobby, won't be big and professional like GNU."
- The Linux kernel was combined with GNU tools to form complete, usable operating systems — often referred to as GNU/Linux.
- Released under the GNU General Public License (GPL), meaning the source code is free to use, study, modify, and share.

Did You Know?

Linux is named after its creator, Linus Torvalds, combined with "Unix," reflecting its Unix-like design. The Linux mascot is a penguin named Tux.

2.4 Timeline at a Glance

Year	Milestone
1969–1970	Unix developed at Bell Labs
1983	Richard Stallman launches the GNU Project
1991	Linus Torvalds releases the first Linux kernel (v0.01)
1992	Linux is relicensed under the GNU GPL
1994	Linux kernel version 1.0 released
1996 onward	Distributions like Red Hat, Debian, and SUSE gain popularity
2004	Ubuntu released, making Linux more accessible to desktop users

Year	Milestone
Today	Linux powers most servers, cloud platforms, and Android devices

2.5 Key People Behind Linux

Person	Contribution
Ken Thompson & Dennis Ritchie	Created Unix at Bell Labs, laying the conceptual foundation for Linux
Richard Stallman	Founded the GNU Project and the Free Software Foundation
Linus Torvalds	Created the Linux kernel and continues to lead its development today
The Open-Source Community	Thousands of contributors worldwide who maintain and improve the kernel and tools

3. Linux Architecture

Linux follows a layered architecture, where each layer interacts with the ones next to it.

Layer	Description
Hardware	The physical components — CPU, RAM, disk, network devices.
Kernel	The core of the OS. Manages hardware resources, memory, processes, and system calls.
Shell	A command interpreter that lets users interact with the kernel through commands (e.g. bash, zsh).
System Utilities / Applications	Tools and programs that perform specific tasks for the user (editors, browsers, compilers).

3.1 The Kernel's Core Responsibilities

- Process management — scheduling and controlling running programs.
- Memory management — allocating and freeing memory as needed.
- Device management — communicating with hardware through drivers.
- File system management — organizing and accessing data on storage devices.

3.2 System Calls

Applications do not talk to hardware directly. Instead, they request services from the kernel through system calls — a defined interface for operations like opening a file, creating a process, or sending data over a network. This separation keeps the system stable and secure, since user programs cannot bypass the kernel to access hardware directly.

4. Features of Linux

Feature	Description
Open Source	Source code is freely available to view, modify, and distribute.
Multuser	Multiple users can access the system and run programs simultaneously.
Multitasking	Can run several processes at the same time without one blocking another.
Portability	Runs on a wide range of hardware, from embedded devices to supercomputers.
Security	Strong permission and user-privilege model reduces the risk of unauthorized access.
Stability	Known for long uptimes and reliability, especially on servers.
Community Support	A large global community continuously improves and supports the ecosystem.

5. Linux Distributions

A Linux distribution (or "distro") bundles the Linux kernel with system tools, package managers, and software to create a complete, ready-to-use operating system.

Distribution	Commonly Used For
Ubuntu	Desktops, beginners, and general-purpose servers
Debian	Stability-focused servers; the base for many other distros
Fedora	Cutting-edge features, desktop use, and development
Red Hat Enterprise Linux (RHEL)	Enterprise servers and business-critical systems
CentOS / Rocky Linux / AlmaLinux	Free, RHEL-compatible server operating systems
Arch Linux	Highly customizable, rolling-release systems for advanced users

Distribution	Commonly Used For
Kali Linux	Security testing and penetration testing

5.1 Choosing the Right Distribution

- Beginners — Ubuntu or Linux Mint, for their ease of setup and large community support.
- Servers — Debian, Ubuntu Server, or RHEL/Rocky Linux, for stability and long-term support.
- Advanced users — Arch Linux, for full control over every installed component.
- Security professionals — Kali Linux, which ships with pre-installed security and testing tools.

6. The Linux File System

6.1 Structure

- Linux organizes all files and directories in a single hierarchical tree, starting from the root directory, written as `/`.
- Unlike Windows, there are no separate drive letters — all storage devices are mounted into this one tree.

6.2 Key Directories

Directory	Purpose
<code>/</code>	The root of the entire file system
<code>/home</code>	Personal directories for each user
<code>/etc</code>	System-wide configuration files
<code>/bin</code>	Essential user command binaries
<code>/var</code>	Variable data — logs, caches, spool files
<code>/tmp</code>	Temporary files, often cleared on reboot
<code>/usr</code>	User-installed software and libraries
<code>/root</code>	Home directory of the root (administrator) user
<code>/dev</code>	Device files representing hardware components

6.3 Absolute vs. Relative Paths

- An absolute path starts from the root directory (/) and specifies the full location of a file, regardless of the current directory.
- A relative path is based on the current working directory, and is often shorter to type.

```
# Absolute path
$ cd /home/student/Documents

# Relative path (from /home/student)
$ cd Documents

# Special path shortcuts
$ cd ..      # move up one directory
$ cd ~      # go to the home directory
$ cd .      # stay in the current directory
```

7. Basic Linux Commands

7.1 Navigation

Command	Purpose
pwd	Print the current working directory
ls	List files and directories
cd	Change the current directory
tree	Display directories as a tree structure

```
$ pwd
/home/student

$ ls -l
$ cd Documents
```

7.2 File and Directory Operations

Command	Purpose
touch file.txt	Create an empty file
mkdir folder	Create a new directory
cp source dest	Copy a file or directory

Command	Purpose
mv source dest	Move or rename a file or directory
rm file.txt	Delete a file
rm -r folder	Delete a directory and its contents

7.3 Viewing and Editing Files

Command	Purpose
cat file.txt	Display the contents of a file
less file.txt	View a file page by page
nano file.txt	Edit a file using the nano editor
vi file.txt / vim file.txt	Edit a file using vi/vim

7.4 System Information

Command	Purpose
whoami	Show the current logged-in user
uname -a	Display system and kernel information
df -h	Show disk space usage
free -h	Show memory usage

7.5 Searching for Files and Text

Command	Purpose
find /path -name file.txt	Search for files by name within a directory tree
grep "text" file.txt	Search for a pattern of text inside a file
grep -r "text" .	Search recursively through all files in a directory
locate file.txt	Quickly find a file using a prebuilt system index

```
$ find /home -name "*.log"
$ grep -r "error" /var/log
```

7.6 Compression and Archives

Command	Purpose
tar -cvf archive.tar folder/	Create an archive from a directory
tar -xvf archive.tar	Extract files from a tar archive
tar -czvf archive.tar.gz folder/	Create a compressed (gzip) archive
zip -r archive.zip folder/	Create a zip archive
unzip archive.zip	Extract a zip archive

7.7 Basic Networking Commands

Command	Purpose
ping google.com	Test connectivity to a remote host
ip a / ifconfig	Show network interfaces and IP addresses
curl url	Fetch the contents of a URL from the command line
wget url	Download a file from the internet
ssh user@host	Connect securely to a remote machine

8. File and Directory Permissions

8.1 Permission Types

- Read (r) — view the contents of a file or list a directory.
- Write (w) — modify a file, or add/remove files within a directory.
- Execute (x) — run a file as a program, or enter a directory.

8.2 Permission Groups

- Owner — the user who owns the file.
- Group — users who belong to the file's assigned group.
- Others — everyone else on the system.

```
$ ls -l notes.txt
-rw-r--r-- 1 student staff 1024 Sep 27 10:00 notes.txt

# Change permissions
$ chmod 755 script.sh

# Change ownership
$ chown student:staff notes.txt
```

Reading Permission Strings

In `-rw-r--r--`, the first character indicates the file type (`-` for file, `d` for directory). The next three groups of three characters represent owner, group, and others' permissions respectively.

8.3 Numeric (Octal) Permissions

Each permission has a numeric value: read = 4, write = 2, execute = 1. These are added together to represent the permissions for owner, group, and others as a three-digit number, used with `chmod`.

Value	Meaning
7	Read + Write + Execute (rwx)
6	Read + Write (rw-)
5	Read + Execute (r-x)
4	Read only (r--)
0	No permissions (---)

```
# rwxr-xr-- => owner: 7, group: 5, others: 4
$ chmod 754 script.sh
```

8.4 Special Permissions

- SetUID — a file runs with the permissions of its owner rather than the user executing it.
- SetGID — a file or new files in a directory inherit the group of the owning group.
- Sticky Bit — in a shared directory, only the file's owner can delete or rename it (used on `/tmp`).

9. Users and Groups

- Linux is a multiuser system — every user has a unique username and user ID (UID).
- Users can belong to one or more groups, which simplifies managing shared permissions.

- The root user (or superuser) has full administrative privileges over the system.

Command	Purpose
adduser username	Create a new user account
passwd username	Set or change a user's password
groups username	Show the groups a user belongs to
sudo command	Run a command with administrative (root) privileges

9.1 Important User-Related Files

File	Purpose
/etc/passwd	Stores basic information about each user account
/etc/shadow	Stores encrypted user passwords, readable only by root
/etc/group	Defines groups and their member users

9.2 su vs. sudo

- **su** — switches to another user account entirely (commonly the root user), requiring that user's password.
- **sudo** — runs a single command with elevated (root) privileges, requiring the current user's own password.
- **sudo** is generally preferred, since it logs actions and avoids staying logged in as root.

10. Process Management

Every running program on Linux is represented as a process, identified by a unique Process ID (PID).

Command	Purpose
ps	List currently running processes
top / htop	Display real-time system resource and process usage
kill PID	Terminate a process by its ID
bg / fg	Send a process to the background or bring it to the foreground

```
$ ps aux | grep firefox
$ kill 4821
```

10.1 Job Control and Priority

Command	Purpose
jobs	List background jobs in the current shell session
command &	Run a command in the background
nice -n 10 command	Start a command with a lower scheduling priority
renice 10 -p PID	Change the priority of an already-running process

11. Package Management

Package managers install, update, and remove software along with its dependencies, saving users from manual setup.

Distribution Family	Package Manager
Debian / Ubuntu	apt (e.g. apt install package-name)
Red Hat / Fedora / CentOS	dnf or yum (e.g. dnf install package-name)
Arch Linux	pacman (e.g. pacman -S package-name)

```
# Update package lists and install a package (Ubuntu/Debian)
$ sudo apt update
$ sudo apt install git
```

11.1 Building Software from Source

When a package is not available through a package manager, software can be compiled directly from its source code. This is less common for everyday use but is a fundamental skill for developers and system administrators.

```
$ ./configure
$ make
$ sudo make install
```

12. Redirection, Pipes, and I/O

Linux treats input and output as streams of data, which can be redirected between files, commands, and devices. This is one of the most powerful ideas in the Linux command line.

12.1 Redirection Operators

Operator	Purpose
>	Redirect output to a file, overwriting its contents
>>	Redirect output to a file, appending to existing contents
<	Redirect a file's contents as input to a command
2>	Redirect error messages to a file

```
$ echo "Hello" > greeting.txt
$ echo "World" >> greeting.txt
$ cat < greeting.txt
$ ls /no/such/folder 2> errors.log
```

12.2 Pipes

The pipe operator (|) sends the output of one command directly into another command as input, allowing several commands to be chained together.

```
$ ps aux | grep bash
$ cat access.log | grep "404" | wc -l
```

13. Shell Scripting Basics

A shell script is a text file containing a sequence of commands that the shell executes in order. Scripts are used to automate repetitive tasks.

13.1 A Simple Script

```
#!/bin/bash
# greet.sh - a simple shell script

name="Student"
echo "Hello, $name! Today is $(date)"
```

13.2 Making a Script Executable

```
$ chmod +x greet.sh
$ ./greet.sh
```

13.3 Conditionals and Loops in Bash

```
#!/bin/bash
count=1

while [ $count -le 3 ]; do
    if [ $count -eq 2 ]; then
        echo "Count is two"
    else
        echo "Count is $count"
    fi
    count=$((count + 1))
done
```

14. Summary

Linux began as a personal project by Linus Torvalds in 1991 and, built on the foundations laid by Unix and the GNU Project, has grown into one of the most widely used operating systems in the world. This documentation has covered its origin and history, its layered architecture of kernel, shell, and applications, and the practical skills needed to navigate it — the file system, essential commands, permissions, user management, processes, package management, I/O redirection, and the basics of shell scripting.

Key Takeaway

Understanding Linux's history and fundamentals provides the groundwork for working confidently with servers, cloud environments, and development systems, where Linux is the dominant operating system.